

Salon Appointment System

Sistem za zakazivanje termina u frizerskom salonu

Projektna dokumentacija

Predmet: Arhitektura softverskih sistema

Student: Kristina Rakić

Jul 2026.

Sadržaj

1. Opis sistema	3
2. Funkcionalni zahtevi	3
3. Nefunkcionalni zahtevi	3
4. Arhitektura sistema	4
4.1 Arhitektonski pristup	4
4.2 Moduli i odgovornosti	4
4.3 Granice modula	5
4.4 Pravila zavisnosti	5
5. C4 dijagrami	6
5.1 System Context dijagram (Nivo 1)	6
5.2 Container dijagram (Nivo 2)	6
5.3 Component dijagram (Nivo 3)	7
6. Dijagram modula i zavisnosti	9
8. Opis implementacije	10
8.1 Složena operacija: Pronalaženje slobodnih termina	10
8.2 Event flow: Kreiranje termina	10
8.3 Zamena perzistencije	10
9. Testiranje	11
10. CI/CD	11
11. Tehnologije i alati	11

1. Opis sistema

Salon Appointment System je WPF desktop aplikacija za zakazivanje i upravljanje terminima u frizerskom salonu. Aplikacija omogućava registraciju klijenata, definisanje usluga sa cenama i trajanjem, upravljanje zaposlenima i njihovim radnim vremenom, zakazivanje i otkazivanje termina sa automatskim pronalaženjem slobodnih slotova, kao i sistem obaveštenja koji reaguje na domenske događaje.

Sistem je dizajniran tako da bude lako proširiv i zamenljiv. Iako je konkretno implementiran za frizerski salon, arhitektura omogućava zamenu teme (stomatolog, auto-servis) promenom podataka bez izmene strukture koda. Perzistencija podataka je zamenljiva, sistem podržava JSON fajlove i SQLite bazu, a prelazak sa jedne na drugu implementaciju je jedna linija u DI registraciji.

2. Funkcionalni zahtevi

Use Case	Opis
Upravljanje klijentima	Kreiranje, izmena i brisanje klijenata sa validacijom unosa.
Upravljanje uslugama	Definisanje usluga sa nazivom, trajanjem i cenom.
Upravljanje zaposlenima	Registracija zaposlenih sa imenom i specijalnošću.
Definisanje radnog vremena	Postavljanje rasporeda po danima u nedelji.
Zakazivanje termina	Pronalaženje slobodnih slotova i kreiranje termina.
Otkazivanje termina	Promena statusa uz emitovanje domenskog događaja.
Pregled rasporeda	Dnevni raspored po zaposlenom sa statusom termina.

3. Nefunkcionalni zahtevi

Zahtev	Opis
Održivost	Modularna struktura omogućava izmenu bez uticaja na ostatak sistema.
Testabilnost	Svi slojevi su testabilni bez UI zavisnosti, sa mock objektima.
Proširivost	Dodavanje nove delatnosti svodi se na promenu podataka.
Zamenjivost	Više implementacija repozitorijuma (JSON i SQLite) kroz DI.

4. Arhitektura sistema

4.1 Arhitektonski pristup

Projekat kombinuje tri arhitektonska pristupa koji se međusobno dopunjuju. Svaki rešava drugi problem - podela sistema, organizacija unutar modula i način povezivanja komponenti.

Modularni monolit

Definiše makro strukturu sistema. Pet funkcionalnih modula (Clients, Services, Staff, Appointments, Notifications) imaju jasne granice i enkapsulaciju. Svi moduli žive u jednom izvršnom procesu ali su logički izolovani. Moduli komuniciraju isključivo kroz SharedKernel, nikad direktno.

Clean arhitektura

Organizuje kod unutar svakog modula. Domain folder sadrži entitete i poslovna pravila, Application folder sadrži use case-ove i servise. Zavisnosti teku ka unutra — Domain ne zna za bazu ni za UI. Princip koji je definisao Robert C. Martin.

Heksagonalna arhitektura (Ports and Adapters)

Definiše način povezivanja komponenti. Interfejsi (portovi) žive u SharedKernel-u (IRepository, IEventDispatcher), a konkretne implementacije (adapteri) u Infrastructure projektu (JsonRepository, SqliteRepository). Zamena adaptera je jedna linija u DI registraciji. Princip koji je definisao Alistair Cockburn.

4.2 Moduli i odgovornosti

Modul	Odgovornost	Zavisi od
SharedKernel	Bazne klase, interfejsi, eventi, Ensure validacija	Ništa
Clients	Registracija i upravljanje klijentima	SharedKernel
Services	Katalog usluga sa cenama i trajanjem	SharedKernel
Staff	Zaposleni i njihovo radno vreme	SharedKernel
Appointments	Zakazivanje, otkazivanje, slobodni slotovi	SharedKernel
Notifications	Reaguje na domenske događaje	SharedKernel
Infrastructure	Repozitorijumi, EventDispatcher, DI	SharedKernel + moduli
WPF	Prezentacioni sloj, MVVM	Svi projekti

4.3 Granice modula

Svaki funkcionalni modul enkapsulira sopstvene entitete i servise. Ne referencira druge module direktno. Appointments modul koristi ClientId, StaffMemberId i ServiceId kao int vrednosti - ne importuje klase iz drugih modula. Jedina međumodulska komunikacija je AppointmentCreatedEvent koji živi u SharedKernel-u i koji Notifications modul osluškuje kroz IDomainEventHandler interfejs. Cikličnih zavisnosti nema.

4.4 Pravila zavisnosti

Zavisnosti teku odozgo nadole — funkcionalni moduli zavise od SharedKernel-a, Infrastructure implementira interfejse iz SharedKernel-a (inverzija zavisnosti — DIP). Nijedan modul ne zavisi od Infrastructure niti od WPF projekta. Domain sloj unutar modula ne zna da postoji baza podataka. Application sloj koristi interfejse (IRepository) bez znanja o konkretnoj implementaciji.

5. C4 dijagrami

5.1 System Context dijagram (Nivo 1)

Prikazuje sistem kao crnu kutiju - ko ga koristi (Klijent, Zaposleni) i gde čuva podatke (SQLite ili JSON).

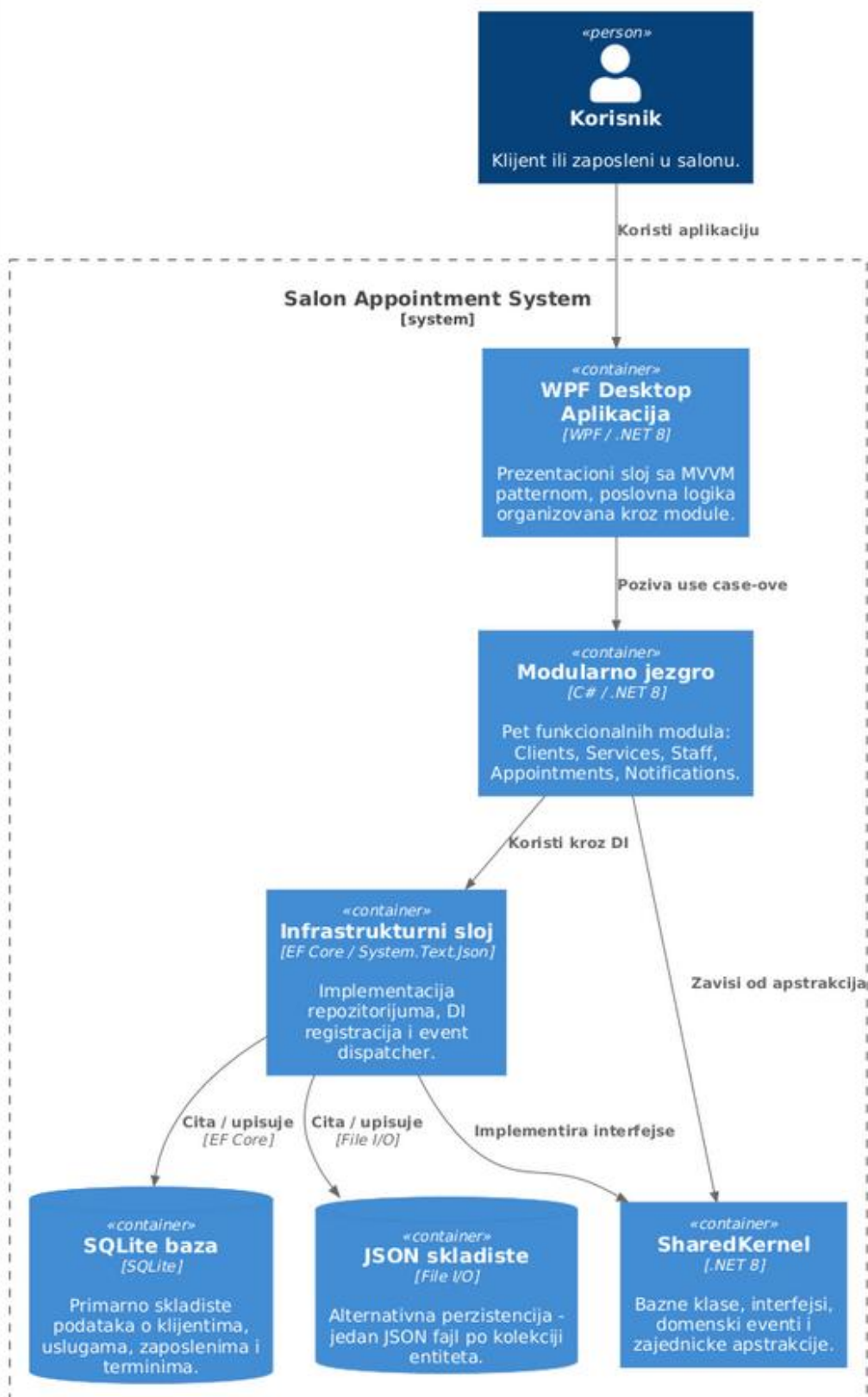
Sistem nema eksternih integracija jer je standalone desktop aplikacija.



5.2 Container dijagram (Nivo 2)

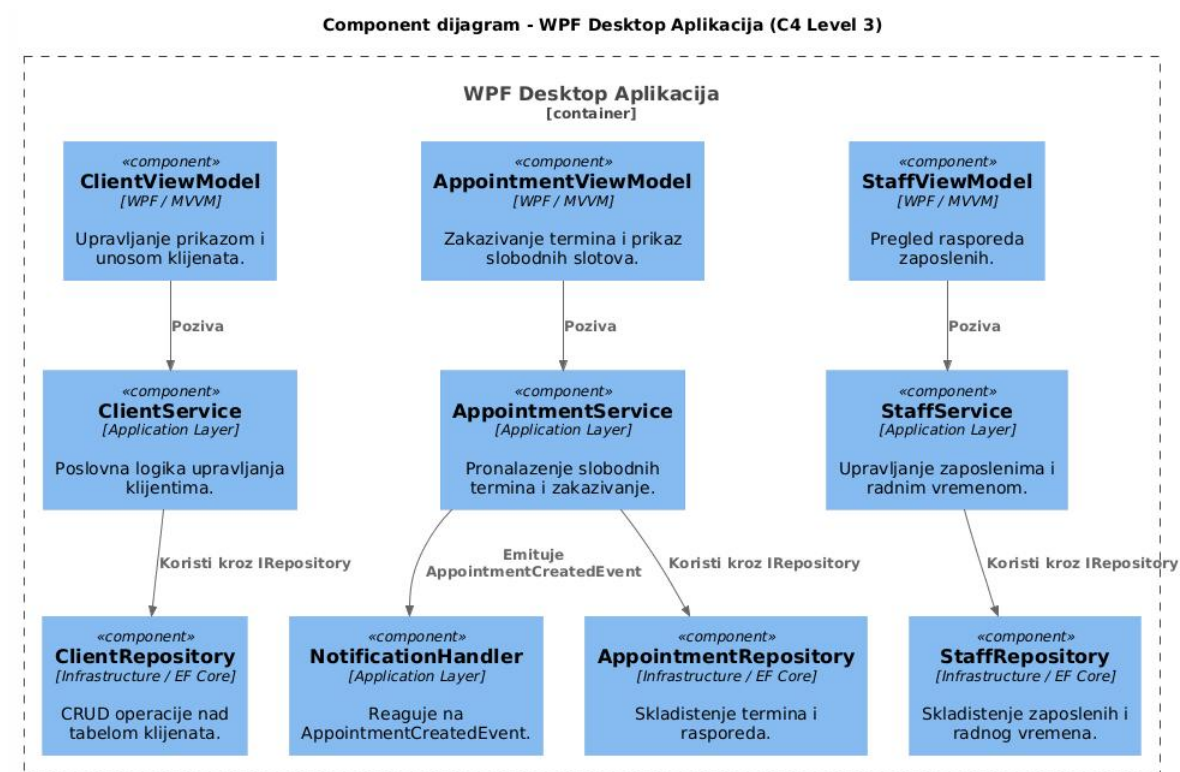
Otvora crnu kutiju iz Context dijagrama i prikazuje tehničke celine sistema - WPF aplikacija sa MVVM patternom, modularno jezgro sa pet funkcionalnih modula, infrastrukturni sloj i dva zamenjiva skladišta podataka (SQLite i JSON).

Container dijagram - Salon Appointment System (C4 Level 2)



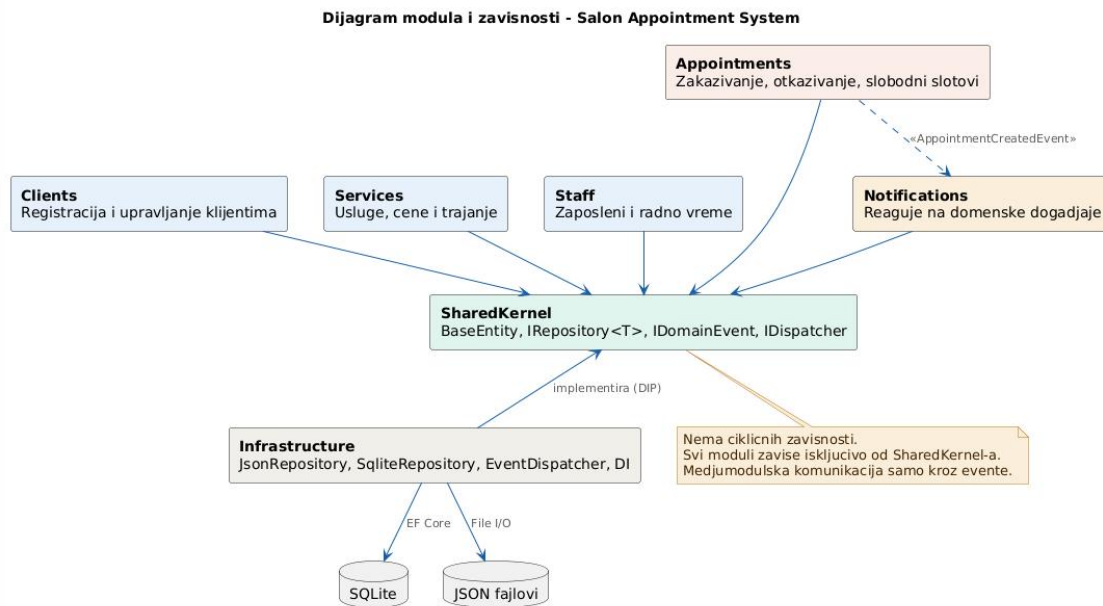
5.3 Component dijagram (Nivo 3)

Prikazuje unutrašnjost WPF kontejnera kroz tri sloja - ViewModels na vrhu, servisi sa poslovnom logikom u sredini, i repozitorijumi na dnu. Komunikacija ide odozgo nadole kroz interfejsa i DI.



6. Dijagram modula i zavisnosti

Prikazuje svih sedam modula i pravce zavisnosti. Funkcionalni moduli zavise od SharedKernel-a, Infrastructure implementira njegove interfejse (DIP), a Appointments komunicira sa Notifications isključivo kroz domenski event. Cikličnih zavisnosti nema.



7. Struktura projekta

```
SalonAppointmentSystem/
├── src/
│   ├── SalonApp.SharedKernel/           - Entity, IRepository, IDomainEvent, Ensure
│   ├── SalonApp.Modules.Clients/         - Domain/Client.cs, Application/ClientService.cs
│   ├── SalonApp.Modules.Services/        - Domain/Service.cs, Application/ServiceManager.cs
│   ├── SalonApp.Modules.Staff/           - Domain/StaffMember.cs, Application/StaffService.cs
│   ├── SalonApp.Modules.Appointments/    - Domain/Appointment.cs, Application/SchedulingService.cs
│   ├── SalonApp.Modules.Notifications/   - Domain/Notification.cs, Application/Handler
│   ├── SalonApp.Infrastructure/          - JsonRepo, SqliteRepo, EventDispatcher, DI
│   └── SalonApp.WPF/                     - ViewModels/, Views/, App.xaml.cs
├── tests/
│   └── SalonApp.Tests/                   - AppointmentTests, SchedulingServiceTests
└── .github/workflows/ci.yml              - GitHub Actions CI pipeline
```

8. Opis implementacije

8.1 Složena operacija: Pronalaženje slobodnih termina

`SchedulingService.FindAvailableSlotsAsync` je centralni algoritam sistema. Prima zaposlenog, datum, trajanje usluge i radno vreme. Učitava sve postojeće termine za tog zaposlenog na taj dan, filtrira otkazane. U petlji od početka do kraja radnog vremena (korak 30 minuta) proverava da li se svaki potencijalni slot preklapa sa postojećim terminom. Ako nema konflikta, dodaje slot u listu slobodnih.

8.2 Event flow: Kreiranje termina

1. Korisnik izabere klijenta, zaposlenog, uslugu, datum i slobodan slot u `AppointmentsView`.
2. `AppointmentsViewModel` poziva `SchedulingService.CreateAppointmentAsync`.
3. `SchedulingService` kreira `Appointment` entitet, sačuva ga kroz `IAppointmentRepository`.
4. `SchedulingService` kreira `AppointmentCreatedEvent` i poziva `IEventDispatcher.DispatchAsync`.
5. `EventDispatcher` pronalazi `AppointmentCreatedHandler` u DI kontejneru.
6. Handler kreira `Notification` entitet i sačuva ga kroz `IRepository`.
7. Ceo tok se dešava bez direktnih referenci između `Appointments` i `Notifications` modula.

8.3 Zamena perzistencije

U `App.xaml.cs`, prelazak sa JSON na SQLite se vrši zamenom jedne linije. Za JSON: `services.AddSalonServices(dataDirectory)`. Za SQLite: `services.AddSalonServicesWithSqlite(path)`. Svi servisi, ViewModeli i entiteti ostaju nepromenjeni jer zavise od `IRepository` interfejsa, ne od konkretne implementacije.

9. Testiranje

Projekat koristi xUnit kao test framework i Moq za kreiranje mock objekata. Testovi pokrivaju Domain i Application sloj bez UI zavisnosti.

Test	Sloj	Šta testira
Cancel_ShouldChangeStatus	Domain	Appointment.Cancel() menja status
Complete_ShouldChangeStatus	Domain	Appointment.Complete() menja status
EndTime_ShouldBeCorrect	Domain	EndTime računa kraj termina
NoBookings_ReturnsAllSlots	Application	Vraća sve slotove kad nema termina
WithBooking_ExcludesSlot	Application	Isključuje zauzete slotove
CreateAppointment_Dispatches	Application	Event se emituje pri kreiranju

Application testovi koriste Mock objekte (Moq) umesto pravih repozitorijuma. Ovo dokazuje da je Dependency Inversion Principle ispravno primenjen — servisi se mogu testirati bez baze podataka.

10. CI/CD

GitHub Actions CI pipeline se automatski pokreće pri svakom push-u na main granu. Pipeline izvršava: checkout koda, setup .NET 8, restore paketa, build celokupnog solution-a i pokretanje svih testova. Ako build ili testovi padnu, pipeline prijavljuje grešku. Konfiguracija se nalazi u `.github/workflows/ci.yml`.

11. Tehnologije i alati

Kategorija	Tehnologija
UI	WPF (.NET 8), XAML, MVVM
Arhitektura	Modularni monolit, Clean Architecture, Hexagonal Architecture
Dependency Injection	Microsoft.Extensions.DependencyInjection
Perzistencija	JSON (System.Text.Json), SQLite (EF Core)
Testiranje	xUnit, Moq
CI/CD	GitHub Actions
Verzionisanje	Git, GitHub